

02. 資料結構視覺化 (Data Structures)

1. Slice 的底層三欄位結構 (Slice Header)

Slice 不是陣列，而是一個包含了三個欄位的結構體（在 64-bit 機器上佔 24 bytes），作為底層陣列的「視窗」。

```
flowchart LR
    subgraph "Slice Header (24 bytes)"
        Ptr["Pointer (8 bytes)"]
        Len["Length (8 bytes)"]
        Cap["Capacity (8 bytes)"]
    end

    subgraph "Underlying Array"
        A0["[0] 10"]
        A1["[1] 20"]
        A2["[2] 30"]
        A3["[3] 空"]
        A4["[4] 空"]
    end

    Ptr --> A0
    Len -. "目前使用到 [2]" .-> A2
    Cap -. "總容量到 [4]" .-> A4
```

2. Append 擴容機制

當 Slice 容量 (cap) 不足時，append 會建立一個新的底層陣列，並將舊資料複製過去，然後回傳新的 Slice Header。

```
sequenceDiagram
    participant S as Slice
    participant O as 舊 Array (Cap: 2)
    participant N as 新 Array (Cap: 4)

    S->>O: append(S, 3) 發現容量已滿
    O-->>N: 1. 分配更大記憶體 (通常 2 倍)
    O->>N: 2. 複製舊資料 [1, 2]
    S->>N: 3. 寫入新資料 [3]
    N-->>S: 4. 更新 Slice Header (新 Ptr, Len:3, Cap:4)
    Note over O: 舊 Array 若無人引用將被 GC 回收
```

3. String 的底層結構

字串 (string) 在 Go 語言中是唯讀的位元組切片 ([]byte)，底層是一個包含兩個欄位的結構體 (16 bytes)。

```
flowchart LR
    subgraph "String Header (16 bytes)"
        Ptr["Pointer to bytes"]
        Len["Length"]
    end

    subgraph "Read-Only Memory"
        B1["G"]
        B2["o"]
        B3["語"]
        B4["言"]
    end

    Ptr --> B1
    Len -. "位元組數 (非字元數)" .-> B4
```

4. strings.Builder 高效拼接

使用 + 號拼接字串會不斷產生新的字串物件並觸發 GC；strings.Builder 則是在一塊預先分配的緩衝區上直接寫入。

```
flowchart TD
    A["s += 'a'"] --> B["建立新字串記憶體"]
    B --> C["複製舊字串與 'a'"]
    C --> D["舊字串變垃圾 (O(N^2) 效能極差)"]

    E["sb.WriteString('a')"] --> F["寫入已分配好的 Buffer"]
    F --> G["若 Buffer 滿了才擴容 (O(N) 效能極高)"]

    style D fill:#f9cfcf,stroke:#ff5555
    style G fill:#c9f9c9,stroke:#55ff55
```